Part 8: Prefill, Decode, and the KV Cache

These notes accompany Part 8: Decode and KV cache. They expand the decode section of my working notes into a guide to the ideas, shapes, and implementation. Part 7 finished the transformer and predicted one next token. This part explains how that becomes a response made of many tokens.

The video-era implementation is commit `7561242`. It handles one request with contiguous key and value allocations. The current source adds batching, persistent scratch buffers, a shared paged cache, and direct paged decode attention. The underlying token timeline is the same. Sections below identify which implementation they describe so the later code does not obscure the lesson.

Read the full video-era files with:

```
git show 7561242:src/main.cpp
git show 7561242:src/kernels.cu
```

For the current implementation, keep `src/main.cpp`, `src/kernels.cu`, and `include/kernels.cuh` open alongside these notes. This engine targets a fixed Llama 3.2 1B Instruct layout; the dimensions below come from its source rather than from a generic model configuration.

1. Two phases of one generation process

Prefill processes the prompt tokens. All of them are already known, so a layer can use matrix operations with one row per prompt token. Causal attention still prevents a position from using a future position.

Decode repeatedly processes the latest generated token to predict another one. For one sequence, only one new input token is available per iteration. That token still goes through every transformer layer.

The word “decode” also appears in the tokenizer interface. `Tokenizer::decode` turns IDs into text. The inference `decode` function performs a transformer forward pass. These are different operations.

For a concrete timeline, let each letter stand for one token ID:

```
Prompt:                   [A, B, C]
Prefill processes:         [A, B, C]       -> predicts D
First decode processes:             D     -> predicts E
Second decode processes:               E  -> predicts F
```

Predicting `D` selects an integer from the vocabulary. It does not run `D` through the transformer. `D` needs its own embedding lookup and forward pass before it can predict `E`.

Completed operation	Tokens processed and cached	Latest predicted ID	Next input position
Prefill	`A, B, C`	`D`	3
Decode `D`	`A, B, C, D`	`E`	4
Decode `E`	`A, B, C, D, E`	`F`	5

Positions are zero based. The cache length counts processed tokens, so it is also the position assigned to the next token we process. The newly predicted ID is one step ahead of the cache.

© 2026 Samir Sarwar. All rights reserved. Personal study permitted. See LICENSE-NOTES.

2. Why earlier positions can be reused

One way to generate would be to run the entire growing prefix again:

```
forward(A, B, C)       -> D
forward(A, B, C, D)    -> E
forward(A, B, C, D, E) -> F
```

That repeats the embeddings, normalizations, projections, attention, and MLP work for `A`, `B`, and `C` every time. Causality gives us a reason to avoid that repetition.

Before appending `D`:

```
A can attend to A
B can attend to A, B
C can attend to A, B, C
```

After appending `D`, those three sets stay the same. Only the new position gets a new attention calculation:

```
D can attend to A, B, C, D
```

For layer 0, earlier positions receive the same embeddings, use the same fixed weights, keep the same absolute positions, and attend to the same allowed prefix. RMSNorm and the MLP process each position independently. Their earlier-position outputs therefore do not need to change.

Layer 1 receives those unchanged earlier-position outputs, so the same reasoning applies again. Following this argument through all 16 layers explains why appending a token does not require recomputing the earlier tokens.

This reasoning assumes the same model, prefix, positions, and inference computation. It does not make an arbitrary cached occurrence of a word reusable in a different prompt: the hidden states and later-layer K/V depend on that word's earlier context.

3. What the cache contains

At every layer, future attention needs earlier **keys and values**. We keep those activation vectors after computing them once.

Item	Stored for future tokens?	Reason
Rotated K vectors	Yes	Future queries compare against these keys.
V projection outputs	Yes	Future attention combines these values.
Earlier Q vectors	No	A future position calculates its own query.
Attention scores or probabilities	No	A different query produces different scores.
`W_k` and `W_v`	Model weights, separate from the cache	These fixed matrices produce the sequence-dependent vectors.
Every intermediate hidden state	No	Future decode attention only needs the earlier K/V from each layer.

© 2026 Samir Sarwar. All rights reserved. Personal study permitted. See LICENSE-NOTES.

The key cache stores **K after RoPE**. The value cache stores **V without RoPE**. Reading an old key does not rotate it again.

Each layer needs its own cache. Layer 5 has different weights and receives different hidden states from layer 0, so layer 0's keys cannot stand in for layer 5's keys.

For one layer after processing `A, B, C`, the logical view is:

```
K: [k_A] [k_B] [k_C]     each row contains 512 BF16 values
V: [v_A] [v_B] [v_C]     each row contains 512 BF16 values
```

Each row contains eight 64-value heads placed beside one another. Query heads are grouped in fours: query heads `0..3` use KV head `0`, query heads `4..7` use KV head `1`, and so on. Integer division implements this mapping:

```
kv_head = query_head / 4
```

We retain eight heads of K/V, not 32 duplicated copies to match the query count. That is one memory benefit of grouped-query attention, or GQA.

4. Track the tensor shapes

Let `T` be the prompt length and `L` the number of tokens already cached before a decode step. The table describes one sequence. In current batched decode, the leading `1` becomes the number of active sequences for projections and MLP operations; each sequence still has its own context length.

Tensor or operation	Prefill	One decode step
Hidden state	`[T, 2048]`	`[1, 2048]`
Q	`[T, 2048] = [T, 32, 64]`	`[1, 2048]`
New K and new V, each	`[T, 512] = [T, 8, 64]`	`[1, 512]`
One layer's valid K or V after writing	`[T, 512]`	`[L + 1, 512]`
Scores for one query head, conceptually	`[T, T]`	`[1, L + 1]`
Attention output across all heads	`[T, 2048]`	`[1, 2048]`
MLP gate and up outputs, each	`[T, 8192]`	`[1, 8192]`
Vocabulary logits used for prediction	`[1, 128256]`	`[1, 128256]`

Prefill needs the hidden states of every prompt token through all layers: later tokens attend to the K/V computed for earlier ones. After the final layer, this generation path only uses the last prompt position for the next-token logits. It does not need a vocabulary projection for every prompt row.

© 2026 Samir Sarwar. All rights reserved. Personal study permitted. See LICENSE-NOTES.

For `L = 3`, one head's decode attention is:

```
q_D_head × K_cached_headᵀ / sqrt(64)
[1, 64] × [64, 4] -> [1, 4] scores

softmax(scores) × V_cached_head
[1, 4] × [4, 64] -> [1, 64] head output
```

The four positions are `A`, `B`, `C`, and `D`. All are allowed by causality. There is no future column to mask in this one-token decode calculation, but reading only the valid prefix is still essential. An allocated slot is not automatically valid context.

5. One decode step, from ID to next prediction

Suppose prefill has processed `A, B, C` and predicted `D`. Before decode, `cache.length == 3`.

1. Look up `D`'s embedding to produce `[1, 2048]`.
2. Assign absolute position `3` to this token.
3. In layer 0, normalize and project the new hidden row into Q, K, and V.
4. Rotate the new Q and K using RoPE position `3`.
5. Write the new K/V into this layer's cache at position `3`.
6. Attend over positions `0..3`, including `D` itself.
7. Apply the output projection, attention residual, second RMSNorm, SwiGLU MLP, and MLP residual.
8. Repeat steps 3 through 7 for layers 1 through 15, using each layer's weights and cache.
9. Apply the final RMSNorm and LM head, then choose the largest logit.
10. After a successful pass, increase the cache length to `4`.

If argmax predicts `E`, the cache now holds `A, B, C, D`. `E` has no cached K/V yet. It becomes the next decode input if generation continues.

The sequence position stays `3` throughout all 16 layers. Entering another layer does not advance the sequence. Incrementing the length inside the layer loop would write different layers into different token slots.

6. RoPE needs the sequence position

The new hidden-state buffer has just one row. Its row index is `0`; its sequence position is `L`. These numbers have different jobs.

Current `decode` constructs positions with:

```
position_ids_cpu[row] = cache->length;
```

For a single sequence with three cached tokens, that uploads `[3]`. Later:

```
launchRope(q_gpu, position_ids_gpu, cos_table_gpu, sin_table_gpu,
           token_count, HIDDEN_SIZE, HEAD_DIM);
launchRope(k_gpu, position_ids_gpu, cos_table_gpu, sin_table_gpu,
           token_count, KV_DIM, HEAD_DIM);
```

These excerpts omit error handling to show the data flow. `token_count` is the number of active decode rows in the current implementation; it is `1` for this example.

© 2026 Samir Sarwar. All rights reserved. Personal study permitted. See LICENSE-NOTES.

Inside `ropeKernel`, `token_index` locates the input row and `position_ids[token_index]` selects the correct sine/cosine table row. Each 64-value head pairs feature `i` with feature `i + 32`, as explained in Part 4.

The previously cached keys already carry their original positions. Reusing them alongside the newly rotated key preserves the positional comparison. Resetting every decode token to position zero, or rotating old keys again, changes that comparison.

7. Video-era code: a contiguous cache

At commit `7561242`, `KVCache` is deliberately simple:

```
struct KVCache
{
    __nv_bfloat16 *keys = nullptr;
    __nv_bfloat16 *values = nullptr;
    int capacity = 0;
    int length = 0;
};
```

Each allocation has logical shape `[16, capacity, 512]`. `initializeKvCache` allocates all those rows up front. `capacity` is reserved space; `length` says how much contains usable results.

The offset to one layer and a new position is:

```
const size_t layer_offset =
    static_cast<size_t>(layer) * cache.capacity * KV_DIM;
__nv_bfloat16 *layer_keys = cache.keys + layer_offset;
__nv_bfloat16 *new_key =
    layer_keys + static_cast<size_t>(position) * KV_DIM;
```

These are offsets in BF16 **elements**, not bytes. The V allocation uses the same indexing.

Prefill projects prompt K/V into temporary buffers, rotates K, and copies those rows into each layer's cache. The valid length becomes `T` after the successful prefill.

Video-era decode instead projects the new K/V directly into their cache row. It rotates `new_key` in place and then runs:

```
computeGqaAttentionScores(..., query_count=1, key_count=L+1)
launchStableSoftmax(..., query_count=1, key_count=L+1)
mixGqaValues(..., query_count=1, key_count=L+1)
```

`computeGqaAttentionScores` and `mixGqaValues` accept a separate key count so attention can be rectangular: one query row against many key rows. The stored score/probability buffer has shape `[32, 1, L + 1]`. No decode causal-mask launch is needed because the prefix contains no future tokens.

That version creates and frees scratch allocations and a cuBLAS handle inside each forward call. Those are temporary working buffers; the K/V allocations survive from prefill until the request finishes. Keeping this distinction clear explains how cached context persists even when local decode scratch is freed.

8. Current code: the same logical cache in pages

The current `KVCache` holds a `KVCachePool*`, a `KVBlockTable`, a capacity, and a valid length. It no longer owns contiguous `keys` and `values` pointers.

© 2026 Samir Sarwar. All rights reserved. Personal study permitted. See LICENSE-NOTES.

One physical page contains K and V for **16 token positions in one layer**:

```
physical page:
[16 × 512 K values][16 × 512 V values]
```

A sequence's block table maps `[layer][logical_page]` to a physical page number in the shared pool. For logical position `p`:

```
logical_page   = p / 16
offset_in_page = p % 16
physical_page  = cache.block_table.layers[layer][logical_page]
```

At position `15`, the new row fits in the first logical page. At position `16`, every layer needs its next page. The physical pages need not be adjacent in memory.

`KVPageAllocator::reserve` checks how many missing pages all layers need before changing the table. Prefill reserves its prompt pages. Decode reserves pages through `length + 1` before writing or uploading the block tables.

`appendKvCache` copies rows from projection scratch into the mapped pages. It handles a partially filled page and crossings between pages, so the same helper supports a whole prompt and a single decode row. It does not increment `length`; the successful outer forward pass commits that update.

`freeKvCache` now returns mapped pages to the allocator and resets the valid length. The shared allocation remains alive for other requests. Old bytes need not be cleared: a new request gets its own page mapping and valid length, and must write every row before reading it.

Current decode calls `launchPagedAttention` to read the pages directly. `gatherKvCache` remains a helper for assembling a dense prefix; it is not part of the active decode attention path. Prefill still uses the dense score, mask, softmax, and value-mixing sequence.

How direct paged attention produces the same result

`pagedAttentionKernel` launches one block per active row and query head, with 64 threads representing the 64 head features. It maps that query head to its GQA KV head, walks only the sequence's valid cached positions, and computes scaled Q·K scores in FP32.

The kernel combines softmax and value mixing using running quantities rather than saving every score. For each score `s` and corresponding value feature `v`:

```
m_new = max(m, s)
correction = exp(m - m_new)
weight = exp(s - m_new)
denominator = denominator * correction + weight
numerator   = numerator   * correction + weight * v
m = m_new

final_feature = numerator / denominator
```

When a larger score arrives, `correction` rescales the previous partial sums to the new maximum. This gives the stable softmax-weighted sum without a `[1, L + 1]` probability buffer. Keys and values remain BF16, the running quantities are FP32, and the final attention output returns to BF16.

Paging changes where K/V live; this online softmax changes how attention is evaluated. Neither changes which earlier positions the new token is allowed to use. Numerical results can still differ from the older BF16 score-buffer path because the rounding points differ.

© 2026 Samir Sarwar. All rights reserved. Personal study permitted. See LICENSE-NOTES.

9. How much memory does reuse cost?

Each processed token contributes K and V at every layer:

```
bytes_per_token = 2 × layers × KV_heads × head_dimension × bytes_per_value
                = 2 × 16 × 8 × 64 × 2
                = 32,768 bytes = 32 KiB
```

For 2,048 valid tokens, the K/V payload is `64 MiB`. With 32 KV heads instead of eight, the payload would be four times larger for the same layer count, head size, and dtype.

This figure excludes model weights, activation scratch, logits, attention scratch, and block-table metadata. The current dense prefill score allocation alone uses `32 × T_capacity² × 2` bytes: at capacity 2,048 that is `256 MiB`. Cache savings do not imply that every other allocation is small.

In the current pool, one physical page occupies:

```
2 × 16 × 512 × 2 = 32,768 bytes = 32 KiB
```

That is a page for one layer, whereas the earlier `32 KiB/token` figure spans all 16 layers. A 16-token span across all layers uses 16 pages, or `512 KiB`. A partly used final page consumes its full physical page space.

The pool defaults to `128 MiB` and is configurable with `--kv-cache-mib N`. Increasing request count does not automatically increase it. `KVCache::capacity == 2048` is the per-sequence logical limit, not a promise that every sequence owns 64 MiB immediately.

The scheduler separately reserves **admission credits** for the maximum prefix a request may process. Physical pages are assigned as that prefix grows. This keeps admitted requests from reaching a later page boundary with no room left to finish; the scheduling details continue in Part 9.

10. Count predictions and stop at the right time

The first prediction comes from prefill and counts toward `max_new_tokens`. A budget of one therefore requires no decode pass.

For prompt length `T` and an allowed output budget `B`, generating all `B` ordinary output tokens requires processing:

```
prompt tokens:             T
generated tokens fed back: B - 1
maximum cached prefix:    T + B - 1
returned output tokens:   B
```

The final predicted token need not be processed because we are not asking for another prediction. For `A, B, C` with budget two, prefill predicts `D`, decode processes `D` and predicts `E`, and generation returns `D, E`. The cache has processed four positions, not five.

Current `generateRequests` uses exactly `T + B - 1` when calculating each request's page budget. Video-era `main` allocated contiguous capacity `T + B`; it reserved one more token slot than the maximum processed prefix needs.

The current CLI clamps the allowed budget:

```
request.token_budget = std::min(
    max_new_tokens, MAX_SEQUENCE_LENGTH - prompt_length);
```

© 2026 Samir Sarwar. All rights reserved. Personal study permitted. See LICENSE-NOTES.

Here `prompt_length` includes the Instruct chat-format tokens produced by `encode_dialog_prompt`, not just the user-visible words. The fixed 2,048-token context contains the prompt and generated response together.

Current `accept_token`, a lambda inside `generateRequests`, handles predictions from prefill and decode consistently. It stops on the tokenizer's EOT or EOS IDs, or `END_OF_MESSAGE_TOKEN_ID` (`128008`, EOM). These stop IDs are not added to the visible generated-token vector or processed in another decode call. The video-era loop checked EOT and EOS; EOM is a later addition.

On ordinary output, the ID is appended and the request continues only while its output count is below its budget. On completion, its pages and admission credits return to the scheduler. Text is decoded from the entire output ID vector so byte pieces split across tokens can join into valid UTF-8.

11. Follow the current code by responsibility

Location / symbol	What to inspect
`src/main.cpp`: `main`	Parse the output/cache settings, chat-format prompts, clamp budgets, and allocate RoPE tables.
`KVCache`, `KVCachePool`, `KVBlockTable`	Separate sequence length/mapping from shared physical storage.
`KVPageAllocator::reserve`, `release`	Assign missing pages and return them on completion.
`prefill`	Reset a request cache, process prompt rows, append each layer's K/V, select the first output ID.
`decode`	Process one row per active sequence using each sequence's absolute position and block table.
`appendKvCache`	Translate logical positions into page offsets and copy K/V rows.
`generateRequests`, `accept_token`	Admit requests, feed predictions back, enforce stopping, and release pages.
`InferenceBuffers`, `freeInferenceBuffers`	Reuse scratch and one cuBLAS handle across layers and forward passes.
`src/kernels.cu`: `ropeKernel`, `launchRope`	Read explicit position IDs when rotating new Q/K.
`pagedAttentionKernel`, `launchPagedAttention`	Read per-sequence pages and compute an online softmax-weighted value sum.
`argmaxKernel`, `launchArgmax`	Pick one vocabulary ID per logit row; exact ties choose the smaller ID.

Most public launch declarations are in `include/kernels.cuh`. In this version, `launchPagedAttention` is instead forward-declared near the top of `src/main.cpp`; do not mistake its absence from the header for an unused kernel.

Current `projection_gpu` holds Q first, then the attention output projection, then the MLP down projection. `normalized_gpu` holds normalization output and later attention output. Reusing these buffers works because the earlier contents have already been consumed. These are temporary activations, separate from the K/V pages that must survive the step.

© 2026 Samir Sarwar. All rights reserved. Personal study permitted. See LICENSE-NOTES.

12. Checks that make a cache bug visible

Run the project commands from the repository root on a supported NVIDIA CUDA machine with the matching model and tokenizer files installed as described in the README:

```
cmake -S . -B build
cmake --build build -j
./build/tokenizer-test models/llama-3.2-1b-instruct/tokenizer.model
./build/samir-vllm --stats "Explain the KV cache in two sentences." 1
./build/samir-vllm --stats "Explain the KV cache in two sentences." 32
```

The tokenizer executable checks tokenizer behavior; it does not verify CUDA attention or KV-cache numerics. A plausible generated response is a smoke check, not a numerical correctness test. No GPU validation result is claimed by these notes.

The one-output-token run exercises prefill and stopping without feedback decode; its `max_decode_batch` statistic should be zero. The larger budget allows decode unless the first prediction is a stop ID. After every completed workload, the reported `final_free_pages` should equal `pool_pages`, and the engine checks that all pages were returned.

For numerical validation, add a small temporary comparison harness around the existing forward functions. Their optional `logits_cpu` output copies FP32 logits back to the host; the normal CLI does not expose a logit-dump option.

Use a fixed list of token IDs and **teacher-force** the same next input ID into each path. Comparing independently generated responses first is harder to diagnose: one slightly different argmax changes the prefix for every later step.

Check	What it should expose
Prefill prefix P, decode a chosen token x, compare with fresh prefill of P + x	Cached attention should represent the same final position as recomputing the whole causal prefix.
Compare against an independent matching model on exactly the same IDs and positions	Catch errors shared by this engine's prefill and decode, such as a common RoPE mistake.
Inspect K/V after prefill and after one decode	Earlier rows should remain unchanged; the new row should appear at the old length in every layer.
Test cached lengths 15, 16, 17, 31, 32, 33	Expose page allocation, offset, and final-page-length errors.
Give a budget of one, then two; force a stop prediction in a harness	Check that prefill's prediction counts and the final/stop ID is not processed again.
Run different prompt lengths together, then reuse a completed slot	Expose position, row-to-cache mapping, or stale-context errors.

Compare intermediate tensors as well as final logits, starting with Q/K/V, RoPE, attention output, and each layer's residual output. Use tolerances informed by BF16 storage and the reduction path; record maximum/relative errors and near-tied top logits. Exact token agreement alone can hide meaningful tensor errors, and a near tie can change argmax despite small tensor differences.

The independent reference must match the checkpoint, chat formatting, RoPE settings, and input IDs. A raw prompt tokenization in one implementation and an Instruct dialog in the other are different inputs. Batched

© 2026 Samir Sarwar. All rights reserved. Personal study permitted. See LICENSE-NOTES.

BF16 GEMMs and the newer FP32 paged attention path can also round differently from their single-request or historical counterparts.

13. Keep these invariants beside the implementation

- Before processing a decode token: `0 < length < capacity`, and its absolute position is `length`.
- The new token's attention includes its own K/V, so it reads `length + 1` valid positions in each layer.
- Every layer writes to the same sequence position, using that layer's own weights and page mapping.
- Cached K is already rotated; cached V is unrotated; old cache entries are never rotated again.
- Scratch row order, position IDs, and block-table row order must describe the same active sequences.
- Valid length advances after a successful full forward pass, not after each layer or after merely predicting an ID.
- A completed request returns its pages before the slot is used for another prompt.

Caching removes repeated computation for earlier positions. It does not remove the new token's MLP or projections, and full-context attention still reads a growing prefix. That trade of GPU memory for reusable context is what makes repeated generation practical and what makes KV memory management central to serving several requests.

© 2026 Samir Sarwar. All rights reserved. Personal study permitted. See LICENSE-NOTES.